

# Nimbus — The Database, Explained

**Version:** 1.5 **Author:** Sujal Kumar Singh **Last updated:** 2026-08-10

What every table is for, in plain English, and why it is shaped that way.

- The exact column list → `HLD.md §8`
- The real SQL that creates it all → `../backend/migrations/versions/`
- Why the system exists at all → `OVERVIEW.md`

The schema is managed by **Alembic migrations**, not a single `schema.sql`. Each change is a numbered, ordered, reversible file. Never edit a migration that has been applied — add a new one. `cd backend && uv run alembic upgrade head` brings any database up to date without deleting the data in it.

## Two files describe this schema, and only one of them creates it

| File                                      | What it is                                                                                   | Who wins                                |
|-------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------|
| <code>backend/migrations/versions/</code> | The SQL that was actually run. A permanent record.                                           | <b>This one.</b> It built the database. |
| <code>backend/src/nimbus/models.py</code> | The same tables as SQLAlchemy classes, so the app can query them without writing SQL strings | Follows the migration                   |

The order matters, and getting it backwards is a real way to lose data. `alembic revision --autogenerate` writes a migration by comparing the models to the live database — but **it cannot see triggers or extensions**. Our refcount trigger (§9, Trap 2) is the only place `blob.refcount` and `mailbox.used_bytes` go down. If someone ever deleted the initial migration and regenerated it from the models, the trigger would vanish without a single error message, refcounts would freeze, and garbage collection would quietly stop reclaiming space.

So the rule is: **the migration leads, the models follow**. To prove they still agree:

```
cd backend
uv run alembic revision --autogenerate -m "drift check"
# the generated file must contain only `pass`
rm migrations/versions/*drift_check.py
```

That empty file is the check. A model that disagrees with the real schema is worse than no model, because everything downstream believes it.

## 1. Three words you need first

| Word               | What it means                                                                                          | What it costs                             |
|--------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <b>Primary key</b> | The column, or columns, that identify a row uniquely. No two rows can share it. It can never be empty. | Nothing — Postgres indexes it for free    |
| <b>Foreign key</b> | A column that must point at a real row in another table. The database refuses to let you break it.     | Checked on every insert and delete        |
| <b>Index</b>       | A shortcut for finding rows. Without one, the database reads the entire table.                         | Slows writes a little, speeds reads a lot |

### Composite keys

A primary key can be **more than one column together**.

```
mailbox_message PRIMARY KEY (mailbox_id, message_id)
```

| mailbox_id | message_id | allowed? |                                   |
|------------|------------|----------|-----------------------------------|
| _____      | _____      | _____    |                                   |
| suja1      | 501        | yes      |                                   |
| ravi       | 501        | yes      | <- same message, different person |
| suja1      | 502        | yes      | <- same person, different message |
| suja1      | 501        | NO       | <- exact pair already exists      |

That single line is what stops the same email landing twice in one inbox.

### What happens when a parent row is deleted

| Rule               | Meaning                                       | Example here                                                      |
|--------------------|-----------------------------------------------|-------------------------------------------------------------------|
| ON DELETE CASCADE  | Children are deleted too                      | Delete a domain → its mailboxes go                                |
| ON DELETE SET NULL | The pointer just becomes empty                | Delete the catch-all mailbox → the domain simply has no catch-all |
| ON DELETE RESTRICT | <b>Refuse the delete</b> while children exist | Delete a reseller that still has blobs → error                    |

## 2. The tables, in four groups

|         |                |                                                                             |
|---------|----------------|-----------------------------------------------------------------------------|
| GROUP 1 | who is who     | reseller, domain, mailbox, alias,<br>shared_mailbox_member, forwarding_rule |
| GROUP 2 | storage engine | blob, chunk, blob_chunk                                                     |
| GROUP 3 | the mail       | message, message_attachment, mailbox_message                                |
| GROUP 4 | supporting     | message_index, provision_order, processed_event                             |

## 3. Group 1 — who is who

### reseller

The **company that resells our email**, not the person reading it.

- Example: "MailHost India" sells mailboxes to their own customers, under their own brand. We are the engine underneath.
- **Multi-tenancy starts here.** Every row in the storage engine is scoped to a reseller.

|              |                                                 |
|--------------|-------------------------------------------------|
| Primary key  | id                                              |
| Unique       | api_key_hash — two resellers cannot share a key |
| Foreign keys | none. This is the top of the tree.              |

### domain

A mail domain one reseller has added. `acme.com`.

|              |                                                                              |
|--------------|------------------------------------------------------------------------------|
| Primary key  | id                                                                           |
| Unique       | name — only one <code>acme.com</code> in the whole system                    |
| Foreign keys | reseller_id → reseller (cascade) · catch_all_mailbox_id → mailbox (set null) |

**There is a loop here.** `domain` points at `mailbox`, and `mailbox` points back at `domain`. Neither table can be created with a foreign key to something that does not exist yet, so that one key is added by `ALTER TABLE` at the end of the file.

### mailbox

One real inbox. `sujal@acme.com` is `local_part = 'sujal'` plus its domain.

| Column        | Why it is there                                               |
|---------------|---------------------------------------------------------------|
| password_hash | Nullable. Shared mailboxes have no login — members read them. |
| used_bytes    | A running total, so checking quota never has to scan messages |
| is_shared     | Routing checks this to decide whether to expand to members    |
| quota_bytes   | What the plan allows                                          |

|             |                                                                                     |
|-------------|-------------------------------------------------------------------------------------|
| Primary key | id                                                                                  |
| Unique      | (domain_id, local_part) — two sujal@acme.com is impossible, sujal@other.com is fine |
| Foreign key | domain_id → domain (cascade)                                                        |

### alias

A nickname pointing at a real mailbox. sales@acme.com → sujal's inbox. Stores no mail of its own.

### shared\_mailbox\_member

Who is allowed to read a shared inbox.

- Primary key is **both columns together**, which by itself stops the same person being added twice.
- This shape is called a **join table** — the standard way to say "many people can be in many mailboxes".
- **Delivery never touches this table.** A shared mailbox receives ONE mailbox\_message row, exactly like any other mailbox; membership decides who may READ it. Fanning a copy out to each member would cost K rows, K refcounts and K quotas for one email, and would give each member a private read state — which is the opposite of what a support@ inbox needs. HLD §9.2.

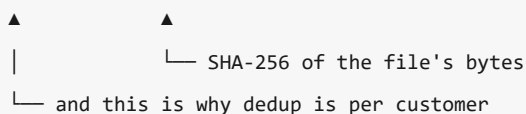
### forwarding\_rule

Send my mail on somewhere else. keep\_copy decides whether we also keep it.

## 4. Group 2 — the storage engine

### blob — one whole attachment

PRIMARY KEY (reseller\_id, hash)



**Why `reseller_id` is in the key.** The same PDF at two different resellers becomes two rows and two stored copies. That looks wasteful and is deliberate: with one global row, an instant upload would tell you that somebody, somewhere, already has that exact file. That is a real published attack against Dropbox. We trade some disk for not leaking across customers.

`refcount` counts `mailbox_message` **rows**, not messages. Counting messages under-counts and frees data two people are still reading.

### `chunk` — a 4 MB piece of a blob

Same composite key. `s3_key` says where the bytes actually live. `refcount` here counts `blob_chunk` rows, so a chunk shared by two blobs survives either one going away.

**Nothing in the database maintains this counter.** The trigger in §9 moves `blob.refcount` and `mailbox.used_bytes` only. `chunk.refcount` is moved by application code — up by the worker when it builds a chunk map, down by the GC worker when it deletes a blob. If that code ever stops doing it, every chunk sits at zero and garbage collection deletes the entire chunk store while it is still being read.

### `blob_chunk` — the ordered map

```
PRIMARY KEY (reseller_id, blob_hash, seq)
```

```
seq 0 → chunk 8e1d..
```

```
seq 1 → chunk b704..
```

```
seq 2 → chunk 2fa9..
```

**`seq` is the whole point.** Chunks come back from S3 in whatever order they arrive. `seq` puts them back together in the right order.

---

## 5. Group 3 — the mail

### `message`

The email itself. **One row, no matter how many people received it.**

### `message_attachment`

Links a message to its files.

**Why `filename` lives here and not on `blob`:** the same bytes might be `invoice.pdf` from one sender and `Q3-final.pdf` from another. The bytes are shared. The name is not.

### `mailbox_message` — the fan-out table

One row per person per email. This is the table the whole design turns on.

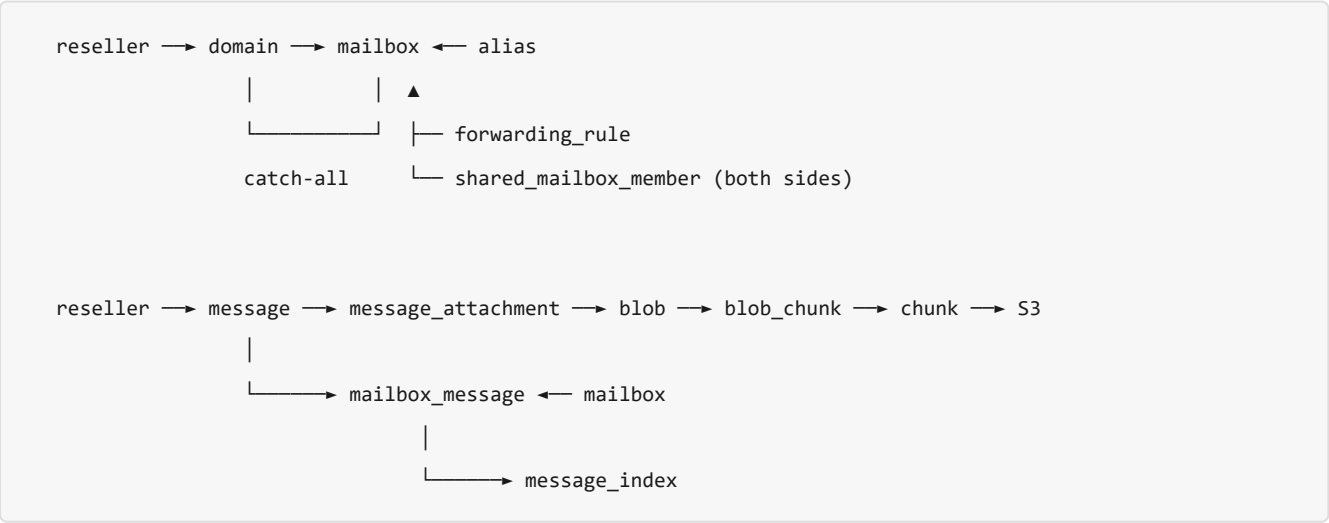
| Holds                                                                  | Why                                                                                                                                                                                                           |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>is_read</code> , <code>folder</code> , <code>snooze_until</code> | <b>Per-person</b> state. Your read/unread is yours, and so is your snooze. There is no <code>is_snoozed</code> flag — snoozed IS <code>snooze_until &gt; now()</code> , so the two cannot disagree (block H). |
| ~40 bytes per row                                                      | 40 recipients cost 1.6 KB, not 40 copies of the attachment                                                                                                                                                    |

Indexed on `(mailbox_id, folder, received_at DESC)` — that serves "show me my inbox, newest first", which is the most-run query in the entire system.

## 6. Group 4 — supporting

| Table                        | What it is for                                     | The key that matters                                                                                                                                                                                                                                         |
|------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>message_index</code>   | Search                                             | One row <b>per delivered copy</b> , keyed <code>(mailbox_id, message_id)</code> . <code>GIN (mailbox_id, tsv)</code> — the mailbox comes first so the tenant filter happens inside the index scan, not after it. Needs the <code>btree_gin</code> extension. |
| <code>provision_order</code> | A record of every provisioning request             | <b>idempotency_key UNIQUE</b> — this one constraint is what makes retries safe                                                                                                                                                                               |
| <code>processed_event</code> | Stops a replayed Kafka event being processed twice | The table exists <i>for</i> its primary key. The collision is the feature, not an error.                                                                                                                                                                     |

## 7. How everything connects



`message_index` hangs off `mailbox_message`, **not** off `message`. That is what makes a deleted copy stop being findable: deleting one `mailbox_message` row cascades to its index row. Hung off `message` instead — which is how it was first built — a deleted message would stay in search results forever, because deleting a message deletes no `message` row at all, only one row of the fan-out table.

---

## 8. What is indexed, and why not everything

---

| Kind            | Where it comes from                            |
|-----------------|------------------------------------------------|
| Automatic       | Every primary key. Every <code>UNIQUE</code> . |
| Written by hand | The list below                                 |

**Postgres does not index foreign keys for you.** Without an index, deleting a parent row means scanning the entire child table to find its children.

But indexing every foreign key is the wrong answer — each index costs a write on every insert. The rule we use: **index a foreign key only where the parent actually gets deleted, and the child table is large.**

| Index                                                                                                               | Why                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>message_attachment</code><br>( <code>reseller_id</code> , <code>blob_hash</code> )                            | <b>Every GC sweep deletes blobs.</b> Hot path.                                                                                                                                                                                                                                                                                                                                       |
| <code>blob_chunk</code> ( <code>reseller_id</code> ,<br><code>chunk_hash</code> )                                   | <b>Every GC sweep deletes chunks.</b> Hot path.                                                                                                                                                                                                                                                                                                                                      |
| <code>mailbox</code> ( <code>domain_id</code> )                                                                     | Deprovisioning a domain                                                                                                                                                                                                                                                                                                                                                              |
| <code>alias</code> ( <code>target_mailbox_id</code> )                                                               | Deleting a mailbox                                                                                                                                                                                                                                                                                                                                                                   |
| <code>shared_mailbox_member</code><br>( <code>member_mailbox_id</code> )                                            | The primary key only covers the other direction                                                                                                                                                                                                                                                                                                                                      |
| <code>forwarding_rule</code><br>( <code>mailbox_id</code> )                                                         | Deleting a mailbox                                                                                                                                                                                                                                                                                                                                                                   |
| <code>mailbox_message</code><br>( <code>message_id</code> )                                                         | The primary key starts with <code>mailbox_id</code> , so it cannot answer "every row for this message"                                                                                                                                                                                                                                                                               |
| <code>message_index</code> ( <code>mailbox_id</code> ,<br><code>tsv</code> ) <b>GIN</b>                             | Search. Both columns, so one index serves free-text, metadata-only and combined queries — GIN, unlike btree, is usable with any subset of its columns                                                                                                                                                                                                                                |
| <code>processed_event</code><br>( <code>message_id</code> )                                                         | Set-null on message delete                                                                                                                                                                                                                                                                                                                                                           |
| <code>mailbox_message</code><br>( <code>mailbox_id</code> , <code>folder</code> ,<br><code>received_at</code> DESC) | Listing an inbox — the most common query                                                                                                                                                                                                                                                                                                                                             |
| <code>message</code> ( <code>thread_id</code> )                                                                     | Conversation view                                                                                                                                                                                                                                                                                                                                                                    |
| <code>blob</code> ( <code>refcount_zeroed_at</code> )<br>WHERE <code>refcount</code> = 0                            | <b>Partial index</b> — only the rows GC cares about, so it stays tiny instead of covering millions of live blobs. On <code>refcount_zeroed_at</code> , <b>not</b> <code>created_at</code> : the sweep asks "how long has this been garbage", and <code>created_at</code> answers "how long ago was it written". Changed in migration <code>9c3e5a1d7b42</code> — see §9 and HLD §9.7 |
| <code>chunk</code> ( <code>created_at</code> ) WHERE<br><code>refcount</code> = 0                                   | Same partial-index idea. <code>chunk</code> needs no zeroed-at clock of its own: a chunk only reaches zero when the last blob referencing it was deleted, and that blob already served the full grace period                                                                                                                                                                         |

**Deliberately not indexed:** `domain.reseller_id`, `blob.reseller_id`, `chunk.reseller_id`, `message.reseller_id`. Those parents are never deleted in normal operation, so the index would be pure write cost.

## 9. Four traps we walked into, and the fixes

These were found by tracing what happens on delete. Both were quiet — nothing would have crashed, the system would just have been wrong.



## Trap 1 — a cascade that orphans S3 forever



So `blob`, `chunk` and `message` use `ON DELETE RESTRICT` on `reseller_id`. Deleting a reseller is a controlled, staged operation. **Failing loudly beats silently orphaning terabytes.**

## Trap 2 — cascades skip the refcount

`blob.refcount` counts `mailbox_message` rows. So every way such a row can disappear must decrement it:

- |                               |                                           |
|-------------------------------|-------------------------------------------|
| 1. a user deletes a message   | application code sees this                |
| 2. a mailbox is deprovisioned | CASCADE — code never sees it              |
| 3. a domain is deleted        | CASCADE of a CASCADE — code never sees it |
| 4. a `message` row is deleted | CASCADE — and the trigger CANNOT HELP     |
| while copies still exist      | see below                                 |

**Path 4 is different from the other three, and the trigger does not save you.** Deleting a `message` row cascades to `mailbox_message` (firing the trigger) *and* to `message_attachment` — and Postgres does not guarantee the order. In practice the attachment rows go first, so the trigger's

```
UPDATE blob ... FROM message_attachment ma WHERE ma.message_id = OLD.message_id
```

joins to nothing, the refcount is never decremented, and that blob sits at `refcount = 1` with no attachment rows and no copies — **invisible to GC's `refcount = 0` filter for ever.** `used_bytes` has the same shape: the trigger's `COALESCE(..., 0)` reads the size from a `message` row that is already gone, so it subtracts nothing and the mailbox's usage stays permanently high. Proven on a scratch replica of the real tables and trigger.

**Nothing reaches path 4 today**, and that is the only reason it is not a live bug. GC phase 1 deletes only messages with zero copies, so the trigger never fires for them, and `DELETE /v1/messages/{id}` removes one `mailbox_message` row rather than a message.

**So it is an invariant, not a guarantee: never delete a message row that still has mailbox\_message rows.** The database will not stop you. Anything that adds bulk deletion, a "purge this mailbox" admin action, or message expiry has to empty the copies first and let the trigger do its work — or the storage engine quietly stops reclaiming.

Paths 2 and 3 happen **inside the database**. Application code is not involved and cannot help. Left alone, refcounts would stay too high forever, GC would never free anything, and the storage engine would quietly stop doing its one job.

**The fix is one trigger on mailbox\_message**, which fires on all three paths including the cascades. It is the only place refcounts go down, and the only place `used_bytes` goes down.

```
DELETE FROM mailbox_message
    |
    ▼
trigger fires (every path, including cascades)
    |
    ├── blob.refcount - 1 for each attachment on that message
    └── mailbox.used_bytes - the message size
```

One place to be right, instead of three places to remember.

### Trap 3 — a search index that outlives what it describes

Found while building block F, and it is the same shape as trap 2: a row that has to disappear when something else does, on paths application code never sees.

`message_index` was originally built with foreign keys to `message` and `mailbox`. Both looked reasonable. Both are useless for the case that actually happens:

```
DELETE /v1/messages/{id}
    |
    ▼
deletes ONE mailbox_message row    ← no `message` row is deleted
                                   ← no `mailbox` row is deleted
    |
    ▼
message_index row survives        ← the message stays searchable FOREVER
    |
    ▼
user searches, finds it, clicks it, gets 404
```

Worse in a shared mailbox: deleting your own copy would leave the team's copy correctly in place but yours orphaned and still matching, so one person's tidying would pollute everyone's search results.

**The fix is a composite foreign key onto the fan-out table**, which is legal because `message_index` and `mailbox_message` share the same key:

```
message_index (mailbox_id, message_id)
  REFERENCES mailbox_message (mailbox_id, message_id) ON DELETE CASCADE
```

The database now maintains it on every path — a user deleting one copy, a mailbox being deprovisioned, a domain cascading, or GC sweeping an orphan. No endpoint has to remember, which is the point: the endpoint that forgets is the one nobody notices for months.

The two original foreign keys were dropped as redundant. That is not just tidying — the new one turns "an index row implies a real delivery" into something Postgres enforces, so a future bug that writes a phantom index row fails loudly instead of producing search results for mail that does not exist.

#### Trap 4 — a `DELETE ... WHERE NOT EXISTS` that does not re-check

Found while building block G's garbage collector, and it is the subtlest of the four, because the SQL looks obviously correct.

Garbage collection deletes messages nobody holds a copy of:

```
DELETE FROM message m
WHERE NOT EXISTS (SELECT 1 FROM mailbox_message mm WHERE mm.message_id = m.id);
```

Read it and it says "only delete a message with no copies". Run it while a delivery is committing, and it deletes a message that **does** have a copy.

| GC                                         | a concurrent delivery        |
|--------------------------------------------|------------------------------|
| —                                          | _____                        |
| DELETE ... WHERE NOT EXISTS                |                              |
| evaluates the subquery: no copies          |                              |
| tries to lock the message row              |                              |
|                                            | INSERT mailbox_message       |
|                                            | takes a key-share lock on    |
|                                            | that same message row        |
| BLOCKS —┐                                  |                              |
|                                            | COMMIT ← the copy now exists |
| unblocks                                   |                              |
| deletes the message ANYWAY                 | X the mail is gone           |
| cascade removes the new copy               |                              |
| trigger drops blob.refcount and used_bytes |                              |

**Why it does not re-check.** When Postgres unblocks, it re-evaluates the statement's conditions against the *row itself* — EvalPlanQual. But this row was never updated, only key-share locked by the foreign key, so there is no new version to re-check against, and the `NOT EXISTS` subquery keeps the snapshot it took before it blocked.

**The fix is two statements instead of one:**

```
SELECT id FROM message m
WHERE NOT EXISTS (...) LIMIT 500 FOR UPDATE SKIP LOCKED;    -- take the lock first

DELETE FROM message                                         -- fresh snapshot,
WHERE id = ANY($1) AND NOT EXISTS (...);                    -- sees the new copy
```

Under READ COMMITTED the second statement takes its own snapshot, which includes anything committed while the first was waiting. `SKIP LOCKED` also means two collectors never fight.

**The same shape as traps 2 and 3.** Each is a correctness rule the database will not enforce for you, and each fails silently in the direction of losing data. This one is the most dangerous of the three because the naive version reads as if it is already safe.

| You want                          | Read                            |
|-----------------------------------|---------------------------------|
| The exact column list             | HLD.md §8                       |
| The real SQL that built it        | ../backend/migrations/versions/ |
| The same schema as Python objects | ../backend/src/nimbus/models.py |
| Why the system exists             | OVERVIEW.md                     |
| The whole system drawn            | ARCHITECTURE.md                 |